

Problem Solving

Introduction

Arnaud Malapert, Marie Pelleau

`arnaud.malapert@univ-cotedazur.fr`

`marie.pelleau@univ-cotedazur.fr`

Acknowledgements

- Wikipedia
- Jean-Charles Régin
- Olivier Bournez, LIX
- Christine Solnon, Université Lyon
- Anne Benoit, ENS Lyon
- Roman Barták, Charles University

Course outline

Lectures

- ① Greedy Algorithms
- ② Local Search

Problem

- A problem is a **general question**: shortest path between two points, timetable
- It is described by data and a question
- Answering this question is solving the problem
- In computer science, we seek a general answer, *i.e.*, an algorithm that works in all cases
- **Instance**: A specific set of data, *e.g.*, the shortest path between Nice and Nantes.

Problem

- Some problems are easy: Sorting numbers, Reversing a string
- Others are difficult: Traveling Salesman Problem (TSP)

Traveling Salesman Problem (TSP)

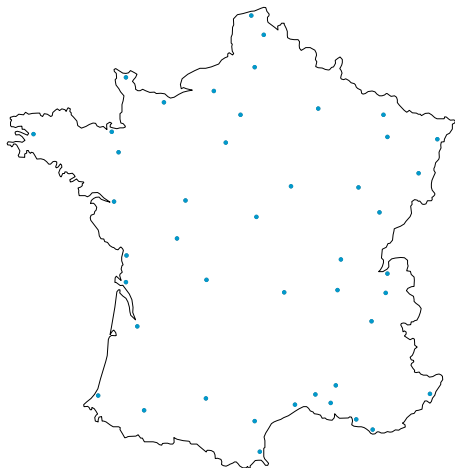
Description

- **Data:** A list of cities and pairwise distances
- **Question:** Find the shortest tour that visits each city exactly once

Mathematical Formulation

Given a complete weighted graph, find a Hamiltonian cycle of minimum weight

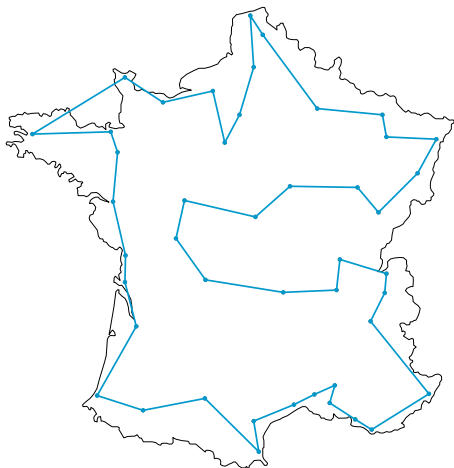
TSP



Description

- All cities are visited exactly once

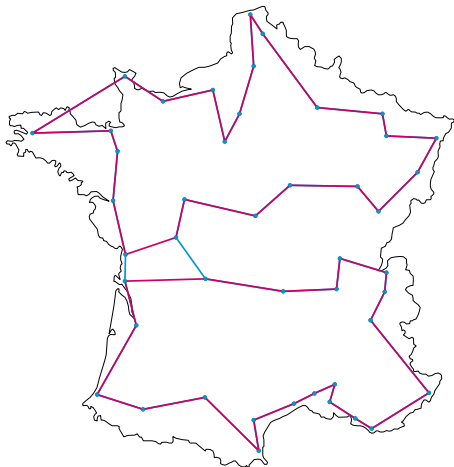
TSP



Description

- All cities are visited exactly once

TSP



Description

- All cities are visited exactly once
- Only one tour (no sub-tours)

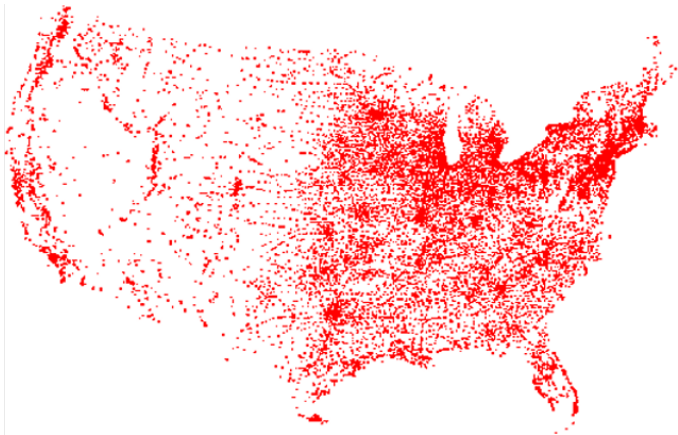
TSP

Description

- Some problems are equivalent to TSP
 - scheduling problems: find the order in which to build objects
- The "pure" version of TSP is rare, in practice often, we encounter variations:
 - Non-Euclidean
 - Asymmetric
- These variations do not make the problem easier
- Common applications
 - Vehicle routing (time windows, pickup and delivery, *etc.*)

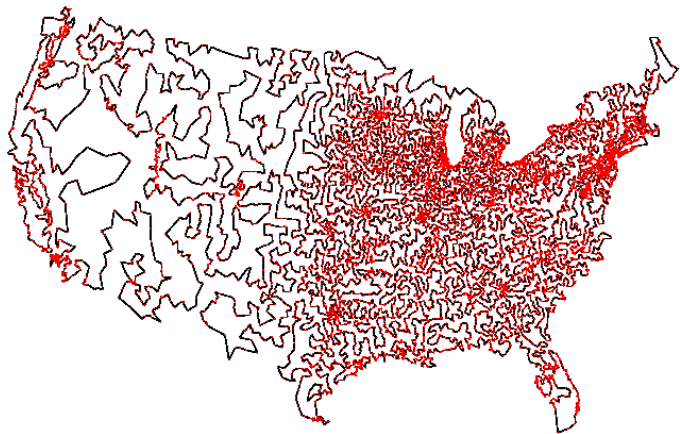
TSP

USA 13 509 cities



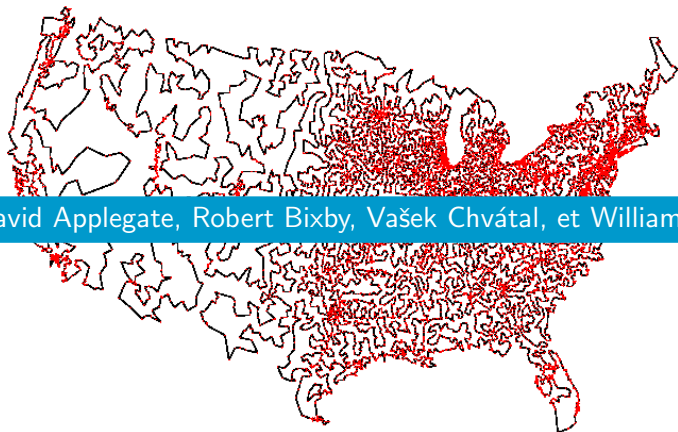
TSP

USA 13 509 cities, solved in 1998



TSP

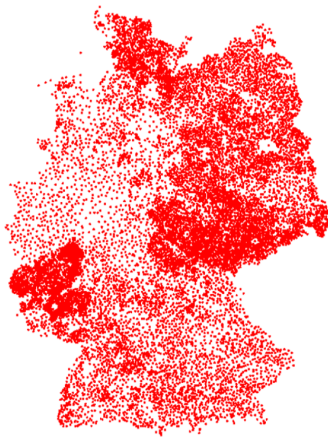
USA 13 509 cities, solved in 1998



By David Applegate, Robert Bixby, Vašek Chvátal, et William Cook

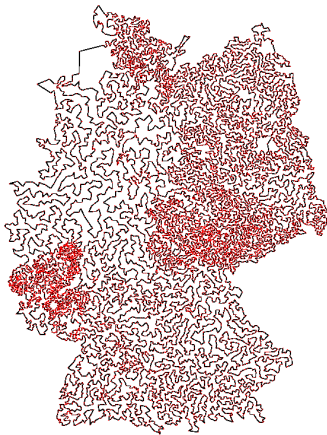
TSP

Germany 15 112 cities



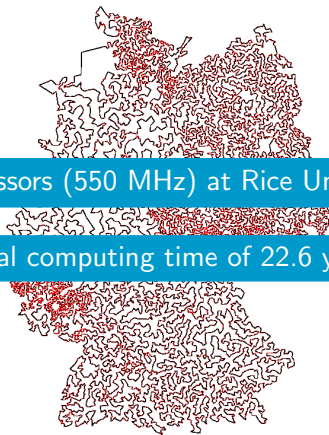
TSP

Germany 15 112 cities, solved in 2001



TSP

Germany 15 112 cities, solved in 2001

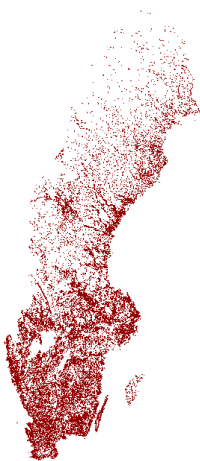


Network of 110 processors (550 MHz) at Rice University and Princeton

Total computing time of 22.6 years

TSP

Sweden 24 978 cities



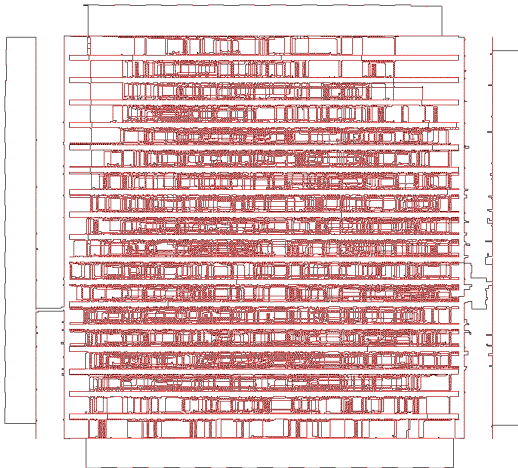
TSP

Sweden 24 978 cities, solved in 2004



TSP

Microchip 85 900 “cities” solved in 2006



TSP

- There exist several solvers
- The most well-known solver is Concorde by William Cook (free)
- TSP solvers are typically dedicated to solving the pure problem
- TSP solvers may not handle even slight variations (asymmetric, additional constraints, *etc.*)

Algorithms

- Not all algorithms are the same, they vary in efficiency, differentiated by:
 - Computation time: slow vs fast
 - Memory usage: low vs high
- We discuss complexity in terms of time (speed) and space (memory used).

Algorithms Complexity

Purpose

- To gauge the difficulty of problems
 - To estimate the computation time or space required to solve a problem
-
- This allows for comparing algorithms
 - Expressed as a function of data size and amount

Problem

Not all problems are the same, they vary according to the algorithms used to solve them

- Easy problem: we know an efficient algorithm to solve it
- Difficult problem: we do not know (yet?) an efficient algorithm to solve it
- Undecidable problem: no algorithm exists to solve it

NP-Completeness

THE major current issue in computer science

P vs NP

- Easy problems have polynomial algorithms
- Difficult problems have no known polynomial algorithm
- **Key question:** Does a polynomial algorithm always exist?

NP-Completeness

For some problems, we do not know if there exists polynomial algorithm, we only know exponential algorithms: 2^n

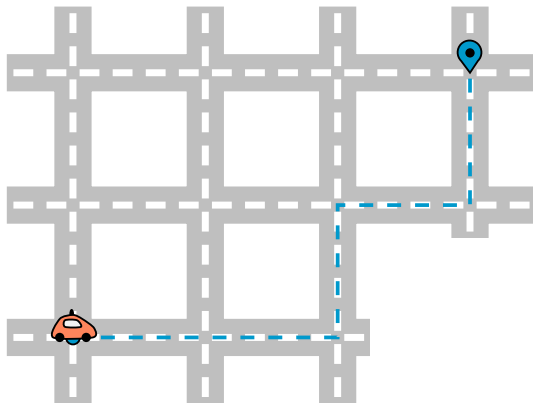
21 NP-Complete Problems by Karp

- Hitting-set: Set covering problem
- Knapsack
- Subset sum
- Bin packing
- Graph coloring
- Maximum clique

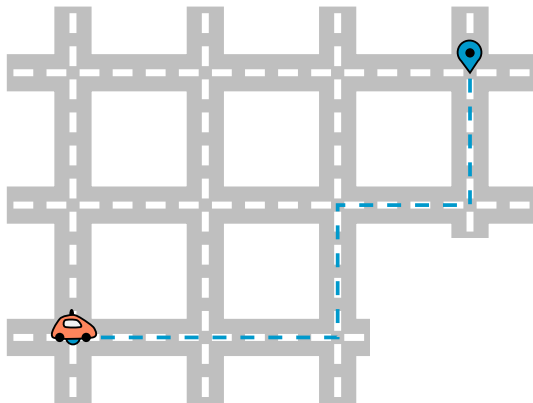
Remark

NP-complete problems are all equivalent and resemble each other. They can be transformed into one another

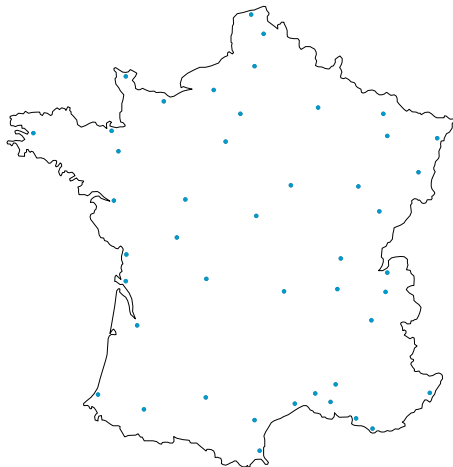
Shortest path



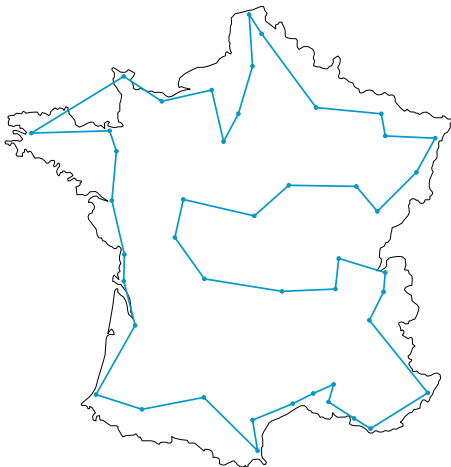
Shortest path – Easy



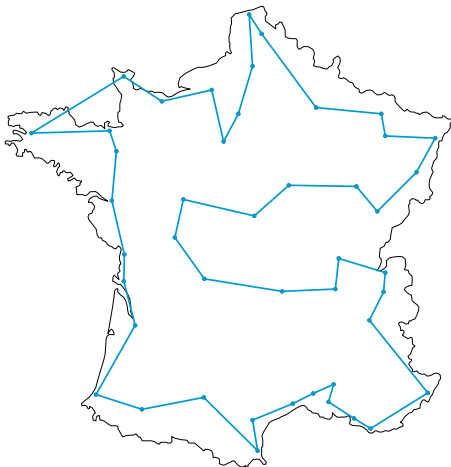
Traveling Salesman Problem



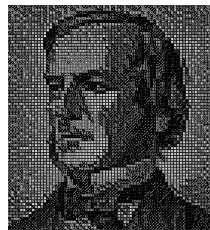
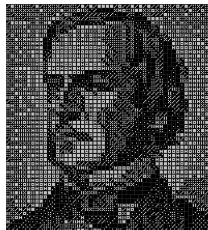
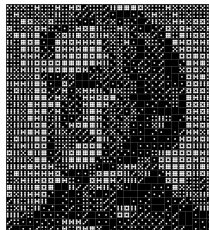
Traveling Salesman Problem



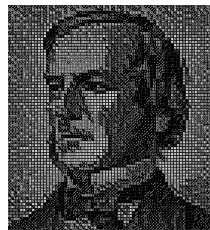
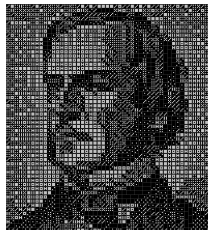
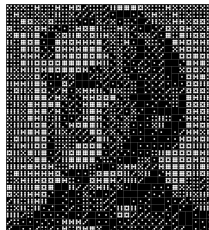
Traveling Salesman Problem – Hard



Dominos protrait



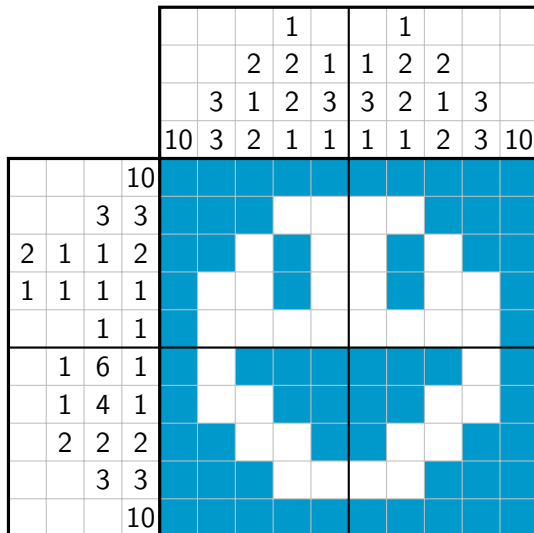
Dominos portrait – Easy



Logigraph

				1		1			
			2	2	1	1	2	2	
		3	1	2	3	3	2	1	3
10	3	2	1	1	1	1	1	2	3
2	1	1	2						
1	1	1	1						
		1	1						
	1	6	1						
	1	4	1						
	2	2	2						
		3	3						
			10						

Logigraph



Logigraph – Hard

[illegible]

Easy vs Difficult

- Given a matrix
- For each row and each column, the number of 1's is known
- Define precisely the 0's and 1's of this matrix

The difference can be subtle

- Pure problem: easy
- connectivity is introduced: difficult
- Convexity is introduced: difficult
- connectivity and convexity are introduced : easy

Decision Problem

A decision problem is a mathematically defined question about given parameters that requires a **yes or no** answer

Example

- Given a set of cities and a distance d , is there a path visiting all cities with a total length less than d ?
- Can a graph be colored with k colors?

Optimization Problem

- An Optimization Problem involves finding the **best solution** among feasible options
- An Optimization Problem has an objective function (min or max)
- An **optimal** solution minimizes (or maximizes) the objective function among all feasible solutions

Example

- Shortest path visiting all the cities?
- Minimum number of colors to color a graph vertices?

Optimization Problem

It is important to distinguish between

- An optimal solution
- Proving that a solution is optimal (optimality proof)

Be careful not to overgeneralize

- Finding optimality and proving it can be slow or fast
- One can be fast and not the other

Decision vs Optimization

Every optimization problem has a corresponding decision problem asking if a solution exists with a particular value

Example

Finding the shortest path between s and t with a cost c

- Decision Problem: is there a path with cost c ?
- Optimality proof: is there a path with cost less than c ?

Decision vs Optimization

Optimization problems are often solved by solving a sequence of decision problems

- We find a feasible solution with cost k
- We ask if there is a solution with cost $< k$ and repeat the process
- At the end, we prove optimality, because the last search does not find a solution.

Hard Problems

- We do not know if hard problems can be quickly solved
- Currently, hard problems cannot be efficiently solved (in polynomial time)
- Current solutions may be inefficient (exponential time)

Approaches Covered in the Course: how to find solutions to problems

- Using heuristics (inexact but fast)
- Complete enumeration of combinations (exact but slow)